

Faculty of Natural Sciences
School of Computer Science and Mathematics

Assessment Brief

Module	CSC-40068
Assessment Component	Coursework
Weighting	100%
Deadline	01:00 pm on 11 September 2023
Module Leader	Amin Noroozi
Office Hours/Meeting Booking Link	Monday and Tuesday during semester two teaching. Email: a.noroozi.fakhabi@keele.ac.uk to arrange a meeting.

1. What is the task for this assessment?

There are Two tasks for this assessment:

- Part1: Programming
- Part II: Code demonstration

2. What is required of me in this assessment?

Guidelines	Detailed guidelines for the tasks are listed below.
Self- assessment checklist Make sure that you...	• The required components and weightings of each part are detailed below. Use these lists to structure your work. • Use the materials discussed in the lectures and tutorials to complete the tasks. • Ensure that your codes run correctly.
Three key pieces of advice based on the feedback given to the previous cohort who completed this assignment	• Read the task description thoroughly and ask for clarification. • Read the formatting guidelines and stick to them. • Start working on your coursework early. Ensure that you understand the tasks and the submission process.
Formatting Guidelines	• Your submission should be in the form of a single zipped file. This zipped file should be named with your Student Number

	- Your zipped file should contain a single .py file for part I.1, a single .py file for part I.2(a), and a single .py file for Part I.2(b) - Highlight the changes you have made to improve your code using concurrency, parallelism and advanced data structures in part I.2(a) and part I.2(b) - All classes and components for each part should be defined in the same .py file, and you shouldn't import them from other files - Your codes should be commented on. It should be specified which part of the code is related to which part of the task
Referencing Style	Harvard referencing style
Assessment Criteria/ Markscheme:	The assessment criteria and mark scheme are listed separately below.

3. What is the purpose of this assessment?

The following table shows which of the module learning outcomes are being assessed in this assignment. Use this table to help you see where and how to transfer feedback from one assignment to another. Note that your feedback may mention some of these outcomes, but that you will not receive a 'mark' against each one.

Module Learning Outcomes assessed
1. apply object-oriented concepts in Python making use of advanced data structures 2. develop software employing parallelism and concurrency 3. justify the extent to which object-oriented programs meet specified requirements

Rationale	- Parts I assess your understanding and ability to implement object-oriented programming, concurrency and parallelism, and the use of advanced data structures - Part II assesses your ability to justify the extent to which your program meets specified requirements
------------------	--

4. What resources might I use to get started?

Lecture slides and your completed in-class activities. However, when needed, you are expected to research and use different resources to complete the tasks.

GUIDELINES

Part I: Pizza Shop Application (90%)

A pizza shop has opened a new branch and needs an application to register the orders. The shop will offer three sizes of Pizza; Small, Medium and Large. An order can contain any combination of the three sizes and may include multiple pizzas of the same size. Each pizza has four attributes, including ID, name, the last time it was ordered, and the ingredients. Each order contains the name or IDs of the pizzas that have been ordered. The shop has storage that can store a maximum of five quantities for each ingredient. Once any ingredient's quantity becomes zero, the shop will buy five new quantities.

The ingredients to cook pizzas are as follows:

- Small Pizza: 1 Dough, 1 Sauce, 2 Toppings
- Medium Pizza: 2 Dough, 1 Sauce, 3 Toppings
- Large Pizza: 3 Dough, 2 Sauces, 4 Toppings

The shop workflow: Once the shop receives an order, they complete the following tasks:

- 1- The order is registered.
- 2- They collect the ingredients for each pizza. If any gradient is out of stock, it will be added to the shopping list.
- 3- The shopping team buy the ingredient in the shopping list if needed.
- 4- Once all gradients are provided, the order is cooked.
- 5- The order is placed into the Collection queue.
- 6- A delivery team collects the order from the collection queue.
- 7- The order is removed from the collection queue.

Registering an order takes 1s. They can collect the ingredients for each order in 1s maximum, and the delivery team can collect an order once every 3s. The shopping list can be bought in 3s. All pizzas take 1s to cook. They may register new orders while completing the previous orders.

PART I.1 [50%]

Using Object-Oriented programming, create an application with GUI that can do the following tasks:

- 1- It could add or remove an order which is a list of pizza names or IDs. It should automatically assign an ID to new orders and print it on the GUI. New orders will initially be assigned a status “registered”. It should have the option to change the orders’ status to “in the collection queue” and “collected”. Once the order has a status of “collected”, it should be automatically removed [7.5%].
- 2- It should calculate the time that each order takes to complete. After the order status is changed to “collected”, it should print a message “The order has been completed in ... hours, ... minutes and ... seconds”. Fill the three dots with the calculated time [5%].
- 3- Given an order ID, it could show the order status [5%].
- 4- It could print the list of all orders, their status, and the order details, including the type and number of pizzas ordered [5%].
- 5- It could generate a shopping list in a PDF format containing the names of the ingredients with zero quantities [5%].
- 6- It could show a timer on the GUI counting the time spent on the current order. It should count the time for all orders from the moment they are registered. But it only shows the timer for the current order [7.5%].
- 7- It could give a report containing the name and ID of the most favourite pizzas and sort them based on how many times they have been ordered. The report should contain the last three times each pizza has been ordered in a PDF format [5%].
- 8- Once an order is updated, it should be stored in a log file. The order log file should include the order ID, time accepted, time collected, number and types of pizzas ordered and their IDs, and the update’s date and time. If any information is missing, it should be entered as None [5%].
- 9- Create a clickable icon for the application. The application should run by clicking the icon without the need to open a Python IDE [5%].

PART I.2- Concurrency, Parallelism and data structures [40%]

(a) Updating the application [20%]

In this part, you are asked to update the application created in PART I. 1 using **the concurrency and parallelism techniques and advanced data structures** discussed in the lecture and tutorial files. You are expected to optimise the program as much as possible. Your updated

application should successfully deal with a hypothetical condition where multiple operatives could use the application simultaneously.

(b) Simulating the shop workflow [20%]

Add a feature to your application to accept 30 orders in a JASON file and simulate the shop workflow. Each order in the JASON file has an ID as its key and the pizza names as its value. You should use concurrency, parallelism, and appropriate data structures. To simulate each task in the workflow, you can write a method to wait for the task's duration and print a message on the GUI upon task completion.

There are a few restrictions on the operations of the shop as follows:

- No order can be put in the collection queue until all pizzas in the order are cooked.
- The shop's storage can keep a maximum of five quantities for each ingredient
- Orders must be collected for delivery in order of their accepted time.

Once all orders from the input order list have been collected, your simulation should end. You should create 30 random orders to test your simulation.

NOTE1: You do not need to create a database for the application for the coursework. Upon starting the program or updating the order details, it could download them from or save them on the hard disk.

NOTE2: Your GUI should have boxes to enter information. An example of the expected shape of GUI is uploaded to the KLE. However, it is just an example, and your application should have specific boxes that will meet the requirements. You might lose marks if you use a GUI that doesn't satisfy the minimum visual requirement shown in the example GUI.

Part II: Code Demonstration (10%)

Once your solutions have been submitted, we will arrange a short demonstration in person or via MS Teams. You will be expected to provide a short explanation of your code and justify how it meets the specified requirements. This demonstration is worth 10% of the module marks.

Assessment Criteria

The criteria are adapted from the University assessment criteria and relate to all parts of this assessment.

Mark

90-100%	Exceptional work that demonstrates an excellent knowledge and understanding of complex issues and methodologies at the forefront of advanced Python OOP programming, concurrency, and parallelism. Contains exceptional evidence of original independent critical thinking that is based upon a sophisticated and rigorous argument. Is accurately supported by evidence derived from a wide range of source material including primary sources and current research.
80-89%	Outstanding work that demonstrates an excellent level of understanding of complex issues and methodologies at the forefront of advanced Python OOP programming, concurrency, and parallelism. Displays independent critical thought, is strong and sophisticated, with well organised argument. Is accurately supported by evidence derived from a wide range of source material including primary sources and current research.
70-79%	Excellent work that demonstrates an excellent level of understanding of complex issues and methodologies at the forefront of advanced Python OOP programming, concurrency, and parallelism. Displays independent critical thought and a strong, organised argument. Is accurately supported by evidence derived from a wide range of source material including primary sources and current research.
60-69%	Very good work demonstrating good understanding of issues related to advanced Python OOP programming, concurrency, and parallelism, including some complex Issues. Displays a good and well organised argument and evaluation with the ability to critically evaluate competing arguments. Is accurately supported by evidence derived from a wide range of source material including primary sources and current research.
50-59%	Good work showing satisfactory grasp of main issues, sufficient awareness of advanced Python OOP programming, concurrency, and parallelism. Argument identified and some analysis of key issues, but with limited critical judgement. Demonstrates sufficient familiarity with a proportion of the basic reading but with minor errors and/or omissions of essential material.
40-49%	Unsatisfactory work showing only limited grasp of some of the issues, poorly conceived and poorly directed to the question or task set, or with serious errors or omissions and limited awareness of the subject or practice. Shows some evidence of planning, although irrelevant/unrelated material or arguments are included. Familiarity with a proportion of the basic reading but with errors and/or omissions of essential material.
30-39%	Unsatisfactory work, showing very limited grasp of some relevant issues and necessary material and/or skills, or with major errors, omissions or misconceptions, and with very limited awareness of advanced Python OOP programming, concurrency, and parallelism. Insufficient attempt to identify argument with irrelevant/unrelated material or arguments included. Evidence of little reading appropriate for the level of study, and/or indiscriminate use of sources.
0-29%	No work offered; or work that is totally irrelevant to the question or task set, fundamentally wrong or shows fragmentary evidence of familiarity with course material or awareness of advanced Python OOP programming, concurrency, and parallelism.